1. Game Documentation

Team Information

- **Student Name :** Basil Khaled Alghamdi.
- **Student ID:** 44104499

General Information

- **Game Title:** Sky Runner
- **Genre:** 2D Platformer / Endless Gatherer
- **Targeted Audience:** Casual gamers of all ages looking for a quick, engaging challenge.

Game Design

- Game Description: Sky Runner is a vibrant 2D platformer where the player controls a character navigating through floating platforms in space. The goal is to collect stars while avoiding lethal bombs that bounce around the stage.
- **Game Play:**
    - Movement: Use the Arrow Keys (Left, Right, Up) to move and jump.
    - Objective: Collect all 12 stars on the screen to reset them and increase your score.
    - Difficulty: Every time you clear a wave of stars, a new bouncing bomb is added to the scene.
    - Game Over: Touching a bomb or falling off the main platforms freezes the game.
- **Motivations for Playing:**
    - High Score: Players are motivated to beat their previous records.
    - Dynamic Difficulty: The game gets progressively harder as more bombs appear.
    - Responsive Controls: Satisfying physics-based movement and jumping.

2. Technical Implementation (Requirements Check)

The game is built using the Phaser 3 framework, fulfilling all the following requirements:

- Scene: Managed through preload, create, and update functions.
- Background Images: Uses high-quality space backgrounds.
- Object Manipulation: Stars are collected (disabled) and respawned dynamically.
- Spritesheets: The player character is an animated spritesheet with walking and idle frames.
- Physics: Utilizes the Arcade Physics engine for gravity, bouncing, and collision detection.
- Input: Integrated Keyboard listeners for real-time control.
- UI/Text: Real-time score tracking displayed on screen.

3. Game Code (index.html)


```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Sky Runner - University Project</title>

<script src="https://cdn.jsdelivr.net/npm/phaser@3.55.2/dist/phaser.js"></script>

  <style>

    body { margin: 0; background-color: #000; display: flex; justify-content: center; align-items: center; height: 100vh; }

  </style>

</head>

<body>

<script>

  const config = {
```

```
  type: Phaser.AUTO,
  width: 800,
  height: 600,
  physics: {
    default: 'arcade',
    arcade: { gravity: { y: 300 }, debug: false }
  },
  scene: { preload: preload, create: create, update: update }
};
let player, stars, bombs, platforms, cursors;
let score = 0;
let scoreText;
let gameOver = false;
const game = new Phaser.Game(config);
function preload() {
  // Preloading Assets
  this.load.image('sky', 'https://labs.phaser.io/assets/skies/space3.png');
  this.load.image('ground', 'https://labs.phaser.io/assets/sprites/platform.png');
  this.load.image('star', 'https://labs.phaser.io/assets/demoscene/star.png');
  this.load.image('bomb', 'https://labs.phaser.io/assets/sprites/bomb.png');
  this.load.spritesheet('dude', 'https://labs.phaser.io/assets/sprites/dude.png',
    { frameWidth: 32, frameHeight: 48 });
```

```
}

function create() {
    // 1. Scene Background
    this.add.image(400, 300, 'sky');
    // 2. Static Platforms (Physics)
    platforms = this.physics.add.staticGroup();
    platforms.create(400, 568, 'ground').setScale(2).refreshBody(); // Base floor
    platforms.create(600, 400, 'ground');
    platforms.create(50, 250, 'ground');
    platforms.create(750, 220, 'ground');
    // 3. Player Sprite & Physics
    player = this.physics.add.sprite(100, 450, 'dude');
    player.setBounce(0.2);
    player.setCollideWorldBounds(true);
    // Character Animations
    this.anims.create({
        key: 'left',
        frames: this.anims.generateFrameNumbers('dude', { start: 0, end: 3 }),
        frameRate: 10,
        repeat: -1
    });
```

```
this.anims.create({ key: 'turn', frames: [ { key: 'dude', frame: 4 } ], frameRate: 20 });

this.anims.create({

  key: 'right',

  frames: this.anims.generateFrameNumbers('dude', { start: 5, end: 8 }),

  frameRate: 10,

  repeat: -1

});

// 4. Input Handling

cursors = this.input.keyboard.createCursorKeys();

// 5. Objects (Stars Group)

stars = this.physics.add.group({

  key: 'star',

  repeat: 11,

  setXY: { x: 12, y: 0, stepX: 70 }

});

stars.children.iterate(function (child) {

  child.setBounceY(Phaser.Math.FloatBetween(0.4, 0.8));

});

bombs = this.physics.add.group();

// 6. UI Implementation

scoreText = this.add.text(16, 16, 'Score: 0', { fontSize: '32px', fill: '#fff' });

// 7. Physics Interactions
```

```
    this.physics.add.collider(player, platforms);
    this.physics.add.collider(stars, platforms);
    this.physics.add.collider(bombs, platforms);
    this.physics.add.overlap(player, stars, collectStar, null, this);
    this.physics.add.collider(player, bombs, hitBomb, null, this);
}

function update() {
    if (gameOver) return;
    if (cursors.left.isDown) {
        player.setVelocityX(-160);
        player.anims.play('left', true);
    } else if (cursors.right.isDown) {
        player.setVelocityX(160);
        player.anims.play('right', true);
    } else {
        player.setVelocityX(0);
        player.anims.play('turn');
    }
    if (cursors.up.isDown && player.body.touching.down) {
        player.setVelocityY(-330);
    }
```

```
}
function collectStar(player, star) {
  star.disableBody(true, true);
  score += 10;
  scoreText.setText('Score: ' + score);
  if (stars.countActive(true) === 0) {
    stars.children.iterate(function (child) {
      child.enableBody(true, child.x, 0, true, true);
    });

    // Logic to drop a bomb
    let x = (player.x < 400) ? Phaser.Math.Between(400, 800) : Phaser.Math.Between(0,
400);
    let bomb = bombs.create(x, 16, 'bomb');
    bomb.setBounce(1);
    bomb.setCollideWorldBounds(true);
    bomb.setVelocity(Phaser.Math.Between(-200, 200), 20);
  }
}
function hitBomb(player, bomb) {
  this.physics.pause();
  player.setTint(0xff0000);
  player.anims.play('turn');
```

```
    gameOver = true;

    this.add.text(200, 250, 'GAME OVER', { fontSize: '64px', fill: '#ff0000' });

  }

</script>

</body>

</html>
```

4. Play/Run Instructions

1. Creation: Save the code above into a file named index.html.
2. Execution: Double-click index.html to open it in any modern web browser (Chrome, Firefox, or Edge).
3. Controls: * Left/Right Arrows: Move character.
    a. Up Arrow: Jump.